

数字电路设计

第 8 章—微体系结构

DDCA Chapter 7: Microarchitecture

【必考章节】

考试重要性：★★★★★

任胜兵老师原话：“考试是必考的”

涵盖内容：

单周期 CPU 设计 · 多周期 CPU 设计 · 流水线处理器 ·
数据通路与控制通路 · 冒险与解决方案 · 高级微体系结构

编译日期：2026 年 6 月 16 日

来源：DDCA ARM Edition Chapter 7 (pp.1–194)

目录

1 微体系结构概述	2
1.1 三种实现方式对比	2
1.2 性能公式（必考）	2
1.3 数据通路 vs 控制通路	2
1.4 结构化建模 vs 行为建模	3
2 性能分析	4
2.1 ARM 指令子集	4
2.2 架构状态元素	4
2.3 性能对比计算实例（必考计算题）	4
3 单周期 CPU 设计【必考】	5
3.1 单周期数据通路——LDR 指令 6 步构建	5
3.2 STR 指令数据通路	6
3.3 数据处理指令数据通路	6
3.4 B 指令数据通路	6
3.5 立即数扩展单元 (ImmSrc)	6
3.6 单周期数据通路完整框图（用 TikZ 绘制）	6
3.7 单周期控制单元	7
3.7.1 控制单元结构	7
3.7.2 Main Decoder 真值表（必考填空！）	7
3.7.3 控制信号含义说明	8
3.7.4 ALU Decoder 真值表	8
3.7.5 PC Logic (PC 逻辑)	8
3.7.6 条件逻辑 (Conditional Logic)	8
3.8 顶层模块结构（结构化建模——必考）	9
3.9 LDR 指令在单周期 CPU 上执行全过程（逐周期描述——必考简答）	9
3.10 LDR 指令周期的关键路径	10
4 多周期 CPU 设计	11
4.1 多周期设计动机	11
4.2 多周期统一数据通路	11
4.3 Main Controller FSM（有限状态机，必考）	11
4.4 各指令所需周期数	11
4.5 多周期 CPI 计算（必考计算）	11
4.6 多周期关键路径	12
4.7 多周期条件逻辑	12

5 流水线处理器设计 【必考】	13
5.1 流水线基本概念	13
5.2 5 级流水线 (IF/ID/EX/MEM/WB)	13
5.3 流水线吞吐率	13
5.4 流水线 vs 单周期 vs 多周期对比	13
5.5 流水线冒险 (Pipeline Hazards)	14
5.5.1 数据冒险 (Data Hazard) —RAW	14
5.5.2 前推 (Data Forwarding) 【必考】	14
5.5.3 停顿 (Stalling)	14
5.5.4 三种数据依赖类型	15
5.6 控制冒险 (Control Hazard) 【必考】	15
5.7 流水线 CPI 计算 (必考)	15
5.8 流水线关键路径	16
6 高级微体系结构	17
6.1 深度流水线 (Deep Pipelining)	17
6.2 微操作 (Micro-operations)	17
6.3 分支预测 (Branch Prediction)	17
6.4 超标量处理器 (Superscalar)	17
6.5 乱序执行 (Out of Order Execution)	17
6.6 寄存器重命名 (Register Renaming)	18
6.7 SIMD (单指令多数据)	18
6.8 多线程与多处理器	18
7 考试重点提示 【必考】 (考前必看!)	19

微体系结构概述

核心概念 1: 微体系结构 (Microarchitecture)

微体系结构是如何在硬件上实现指令集架构 (ISA) 的具体方式。同一架构 (如 ARM) 可以有多种微体系结构实现。

三种实现方式对比

表 1: 三种处理器实现方式对比

特性	单周期 (Single-cycle)	多周期 (Multicycle)	流水线 (Pipelined)
执行方式	每条指令在一个周期内完成	每条指令分解为若干短步骤，不同指令需要不同周期数	每条指令分解为若干步骤，多指令同时执行
CPI	固定为 1	加权平均 (如 4.12)	理想为 1，实际 1.23
时钟周期	由最长指令决定 (如 840ps)	较短 (如 340ps)	更短 (如 300ps)
硬件复用	不可复用，需 3 个 ALU/加法器	可复用硬件	不可复用，每个阶段有独立硬件
优点	设计简单	时钟快，硬件复用	吞吐率最高
缺点	周期长，资源浪费	有排序开销	有冒险 (hazard)

性能公式 (必考)

核心概念 2: 处理器性能公式

执行时间 = #instructions × CPI × T_c (1)

其中:

- CPI (Cycles Per Instruction): 每条指令平均时钟周期数
- T_c (Clock Period): 时钟周期 (秒/周期)
- $IPC = 1/CPI$: 每周期执行指令数

考试中会给延迟参数，要求计算 T_c 和执行时间，必须掌握！

数据通路 vs 控制通路

核心概念 3: 数据通路 (Datapath)

对指令进行数据处理的功能模块组合，以字 (32-bit) 为单位处理。包括：PC、寄存器堆、ALU、多路选择器、存储器。

核心概念 4: 控制通路 (Control Path)

产生控制信号的电路，主要控制多路选择器 (MUX) 选择正确的数据通路。

考试常考：哪些属于数据通路？哪些属于控制通路？控制信号控制什么？

结构化建模 vs 行为建模

- 结构化建模 (Structural Modeling): 用底层模块实例化的方式搭建系统，体现硬件结构
- 行为建模 (Behavioral Modeling): 用 always 块描述功能行为
- 考试中单周期 CPU 顶层模块要求结构化建模！

性能分析

ARM 指令子集

本课程考虑的 ARM 指令子集：

- 数据处理指令：ADD, SUB, AND, ORR（寄存器和立即数 Src2，不含移位）
- 存储器指令：LDR, STR（正立即数偏移）
- 分支指令：B
- 比较指令：CMP（扩展功能）

架构状态元素

核心概念 5: 架构状态 (Architectural State)

决定处理器一切行为的状态：

- 16 个寄存器 (R0–R15)，其中 R15 为 PC
- 状态寄存器 (Flags[3:0] = NZCV)：N(负)、Z(零)、C(进位)、V(溢出)
- 存储器 (Memory)

性能对比计算实例（必考计算题）

教材示例：1000 亿条指令程序在三种处理器上的执行时间对比。

元件延迟参数（用于所有计算）

表 2: 元件延迟参数		
元件	参数	延迟 (ps)
寄存器 clock-to-Q	t_{pcq_PC}	40
寄存器 setup	t_{setup}	50
多路选择器	t_{mux}	25
ALU	t_{ALU}	120
译码器	t_{dec}	70
存储器读	t_{mem}	200
寄存器堆读	t_{RFread}	100
寄存器堆 setup	$t_{RFsetup}$	60
寄存器堆写	$t_{RFwrite}$	70

三种实现性能对比表

注意：多周期虽然在时钟周期上更优，但由于 CPI 高，实际执行时间反而比单周期更长（140 vs 84 秒）。流水线取得了 2.28 倍加速比。这可能是考试分析题！

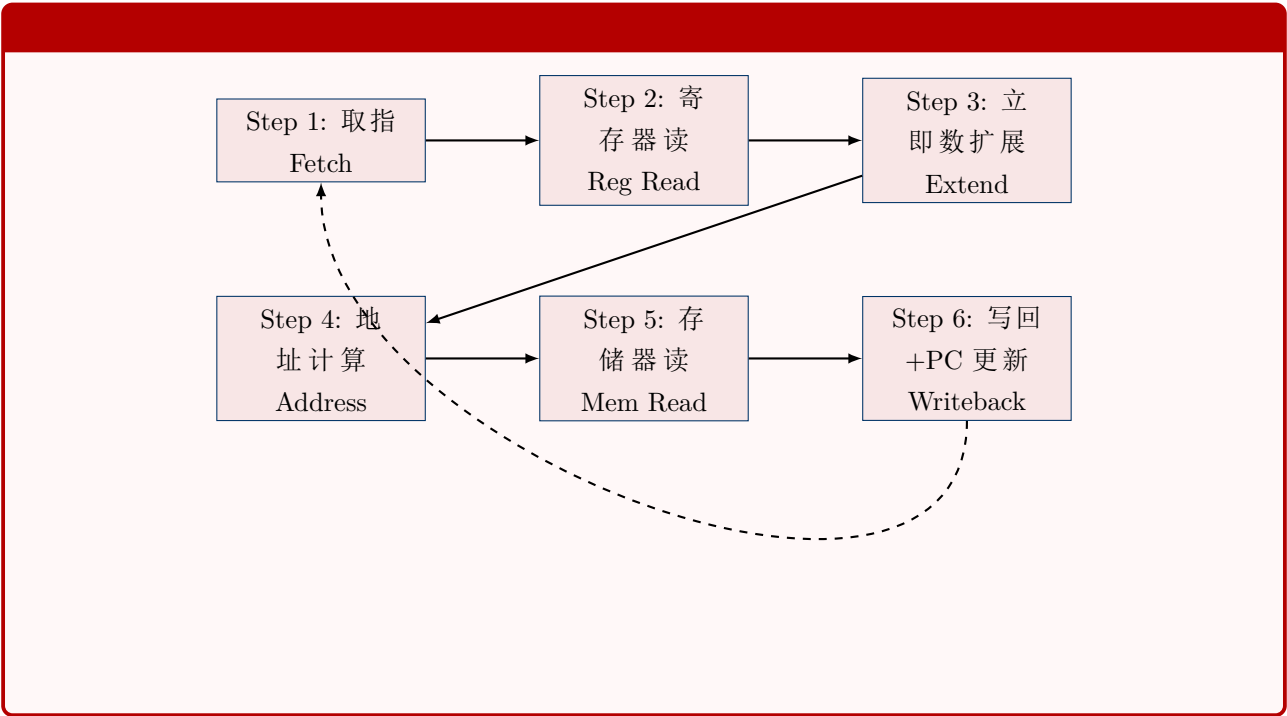
表 3: 三种实现性能对比（1000 亿条指令）

处理器	T_c (ps)	CPI	执行时间 (s)	加速比
单周期	840	1	84	1（基准）
多周期	340	4.12	140	0.6
流水线	300	1.23	36.9	2.28

单周期 CPU 设计【必考】

单周期数据通路——LDR 指令 6 步构建

单周期数据通路以 LDR Rd, [Rn, imm12] 为例，分 6 步构建。



Step 1 —Fetch (取指)

PC → 指令存储器地址 → 读出指令。PC = PC + 4。

Step 2 —Register Read (寄存器读)

从寄存器堆读取 Rn(源寄存器)。读口 A1=Instr[19:16](Rn)。

Step 3 —Immediate Extend (立即数扩展)

将 12 位立即数 imm12 零扩展到 32 位：{20'b0, Instr[11:0]}。

Step 4 —Address Compute (地址计算)

ALU 计算存储器地址：ALUResult = Rn + ExtImm。

Step 5 —Memory Read (存储器读)

以 ALUResult 为地址，读数据存储器，得到 MemData。

Step 6 —Writeback + PC Update (写回)

将 MemData 写入寄存器堆 Rd(Instr[15:12])。确定下一条指令地址。

STR 指令数据通路

STR 与 LDR 的区别:

- LDR: 从存储器读数据 → 写回寄存器堆。MemtoReg=1, MemW=0。
- STR: 将寄存器数据写入存储器。需要从 Rd 读取源数据，MemW=1。

STR 数据通路差异:

1. 步骤相同: Fetch → Reg Read → Extend → Address
2. 不同点: Rd(Instr[15:12]) 的值作为写数据 (WD) 送到数据存储器
3. 寄存器堆需要两个读口: RA1=Rn, RA2=Rd
4. 地址计算方式相同: Rn + imm12

数据处理指令数据通路

立即数 Src2 (ADD Rd, Rn, imm8) • SrcA = Rn, SrcB = 零扩展 imm8

- ALUResult = Rn + ExtImm8
- 写回 Rd: WD3 = ALUResult, WA3 = Rd
- ImmSrc = 00 (选择零扩展 imm8)

寄存器 Src2 (ADD Rd, Rn, Rm) • SrcA = Rn, SrcB = Rm

- ALUResult = Rn + Rm
- 写回 Rd: WD3 = ALUResult, WA3 = Rd
- RegSrc 选择两个寄存器读口

B 指令数据通路

分支目标地址计算:

$$BTA = (ExtImm24) + (PC + 8)$$

(2)

其中 ExtImm24 = Imm24 << 2 并符号扩展。

ImmSrc = 10: {6{Instr[23]}, Instr[23:0]} (符号扩展 24 位立即数并左移 2 位)

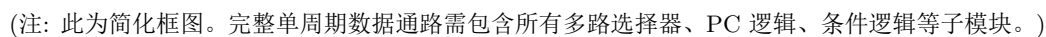
立即数扩展单元 (ImmSrc)

表 4: 立即数扩展编码

ImmSrc[1:0]	扩展方式	用途
00	{24'b0, Instr[7:0]} —零扩展 imm8	DP 立即数
01	{20'b0, Instr[11:0]} —零扩展 imm12	LDR/STR
10	{6{Instr[23]}, Instr[23:0]} —符号扩展 imm24	B 指令

考试填空常考: 不同指令的立即数扩展位数 (8→32, 12→32, 24→32) 和扩展方式!

单周期数据通路完整框图 (用 TikZ 绘制)



这张真值表是必考填空/简答内容！需要理解每个控制信号的含义及各指令类型下的取值。

表 6: 主要控制信号含义

信号	含义与作用
MemtoReg	写回数据选择: 0=ALUResult, 1=MemData (LDR 时 =1)
MemW	存储器写使能: 1= 写存储器 (STR 时 =1)
ALUSrc	ALU 第二操作数选择: 0= 寄存器, 1= 立即数
ImmSrc[1:0]	立即数扩展类型选择: 00=imm8, 01=imm12, 10=imm24
RegW	寄存器堆写使能: 指令需写回 Rd 时为 1
RegSrc[1:0]	寄存器读口地址选择
ALUOp	ALU 操作使能: DP 指令 =1, 非 DP 指令 =0
Branch	分支指令标志: B 指令 =1
PCSrc	PC 写选择: 1= 写入 BTA 或 ALUResult, 0=PC+4

表 7: ALU Decoder 真值表【必考】

ALUOp	Funct[4:1]	Funct0	Type	ALUControl[1:0]	FlagW[1:0]	NoWrite
0	X	X	Not DP	00 (Add)	00	0
1	0100	0	ADD	00 (Add)	00/11	0
1	0010	0	SUB	01 (Subtract)	00/11	0
1	0000	0	AND	10 (AND)	00/10	0
1	1100	0	ORR	11 (OR)	00/10	0
1	1010	1	CMP	01 (Subtract)	11	1

3.7.3 控制信号含义说明

3.7.4 ALU Decoder 真值表

FlagW 含义: FlagW[1]=1: 保存 NZ 标志 (Flags[3:2]); FlagW[0]=1: 保存 CV 标志 (Flags[1:0])。ADD/SUB 更新所有标志 (FlagW=11); AND/ORR 仅更新 NZ(FlagW=10)。

3.7.5 PC Logic (PC 逻辑)

PCS 信号产生:

$$PCS = ((Rd == 15) \& RegW) \mid Branch \tag{3}$$

即: 当指令写 R15(PC) 或是分支指令时, PCS=1。

PCSrc = PCS (若 CondEx=1), 否则 PCSrc=0 (PC=PC+4)。

3.7.6 条件逻辑 (Conditional Logic)

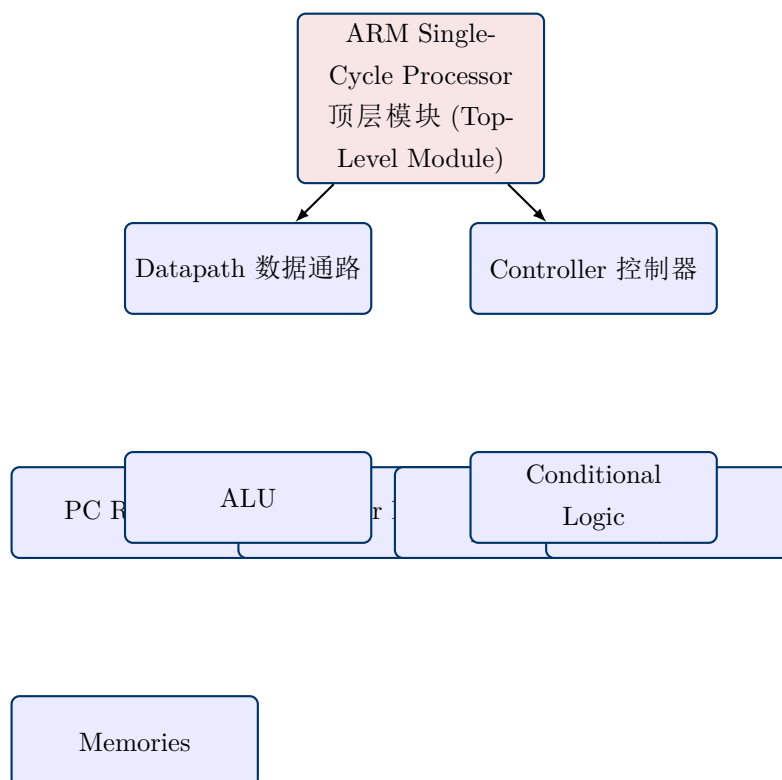
条件逻辑功能:

- 1. 条件执行判断: 根据 Cond[3:0] 和 Flags[3:0], 决定指令是否执行 (CondEx=1)
- 2. 强制信号为 0: 若 CondEx=0, 强制 PCSrc、RegWrite、MemWrite 为 0
- 3. 更新状态寄存器: 若 FlagW 非零且 CondEx=1, 更新 Flags

考试可能问：ALUFlags 何时更新？答：当 FlagW 非零且 CondEx=1 时。

顶层模块结构（结构化建模——必考）

单周期处理器顶层模块结构：



Verilog 结构化建模关键点：

- 顶层模块实例化 datapath 和 controller 子模块
- 通过 wire 连接控制信号
- 不写任何 always 块（行为建模相反）

LDR 指令在单周期 CPU 上执行全过程（逐周期描述——必考简答）

以 LDR R1, [R2, #5] 为例 ($R_d=R1$, $R_n=R2$, $imm12=5$):

1. 取指 (Fetch): PC 值作为地址送指令存储器, 读出 32 位指令码。PC 同时更新为 $PC+4$ 。
2. 译码 + 寄存器读 (Decode & Reg Read): 主译码器解析指令类型为 LDR。RegSrc 选择 $RA1=R_n(R2 \text{ 地址})$ 。寄存器堆读出 R2 的值。
3. 立即数扩展 (Extend): $ImmSrc=01$, 12 位立即数 #5 零扩展为 32 位: $\{20'b0, 12'd5\}$ 。
4. 地址计算 (Execute): $ALUSrc=1$ (选立即数), ALU 执行加法: 地址 = $R2 + 5$ 。
5. 存储器读 (Memory): 以上一步计算的地址读数据存储器。MemtoReg=1。
6. 写回 (Writeback): $RegW=1$, 读出的数据通过 Result 多路选择器写入寄存器堆 $WA3=R_d(R1 \text{ 地址})$ 。更新 Flags (若 $S=1$)。
7. PC 更新: Branch=0, $R_d \neq 15$, $PCSrc=0$, $PC=PC+4$ (取下一条指令)。

关键控制信号时序: $Op=01$, $Funct0=1 \rightarrow MemtoReg=1$, $MemW=0$, $ALUSrc=1$, $ImmSrc=01$, $RegW=1$, $ALUOp=0$ 。

LDR 指令周期的关键路径

单周期处理器周期时间由最长指令 LDR 的关键路径决定：

$$T_{c1} = t_{pcq_PC} + t_{mem} + t_{dec} + \max[t_{RFread} + t_{sext}, t_{mux}] \\ + t_{mux} + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup} \quad (4)$$

代入延迟参数：

$$T_{c1} = 40 + 2(200) + 70 + 100 + 120 + 2(25) + 60 = 840 \text{ ps} \quad (5)$$

多周期 CPU 设计

多周期设计动机

单周期的缺点：

- 周期时间由最长指令 (LDR) 决定，其他快速指令浪费时钟周期
- 需要独立的指令存储器和数据存储器（哈佛架构）
- 需要 3 个 ALU/加法器

多周期改进：

- 将指令分解为更短的步骤，不同指令使用不同步骤数量
- 更高的时钟频率
- 硬件复用（如统一存储器、复用 ALU）

多周期统一数据通路

多周期使用**统一存储器**（单一存储器同时存放指令和数据），更贴近真实硬件。

通过添加**内部寄存器**（非架构寄存器）保存中间结果：

- IR (指令寄存器): 保存当前指令
- ALUOut: 保存 ALU 中间结果
- Data: 保存存储器读数据

Main Controller FSM（有限状态机，必考）

多周期控制通过 **FSM** 实现，考试常考状态转移和每状态的控制信号。

多周期 FSM 状态描述：

表 8: 多周期 FSM 状态与指令周期数

状态	用途	说明
S0: Fetch	取指 (所有指令)	$IR = Mem[PC]$; $PC = PC + 4$
S1: Decode	译码 + 寄存器读 (所有指令)	读 R_n/R_m ; 扩展立即数; 计算 BTA
S2: MemAdr	地址计算 (LDR/STR)	$ALUOut = R_n + ExtImm$ (仅 LDR/STR)
S3: MemRead	存储器读 (LDR)	$Data = Mem[ALUOut]$ (仅 LDR)
S4: MemWB	写回 (LDR)	$Reg[Rd] = Data$ (仅 LDR)
S5: MemWrite	存储器写 (STR)	$Mem[ALUOut] = Rd$ (仅 STR)
S6: ExecuteI	执行立即数 DP	$ALUOut = R_n + ExtImm$; 写回 Rd
S7: ExecuteR	执行寄存器 DP	$ALUOut = R_n + R_m$; 写回 Rd
S8: ALUWB	写回 DP	$Reg[Rd] = ALUOut$; 更新 Flags
S9: Branch	分支	$PC = BTA$ (若条件满足)

各指令所需周期数

(考试常考：给出指令类型，问需要几个周期)

多周期 CPI 计算（必考计算）

SPECINT2000 基准测试指令分布：

- 25% loads (LDR)

表 9: 各指令周期数

指令类型	周期数	状态序列
B	3	Fetch → Decode → Branch
DP (数据处理)	4	Fetch → Decode → ExecuteI/ExecuteR → ALUWB
STR	4	Fetch → Decode → MemAdr → MemWrite
LDR	5	Fetch → Decode → MemAdr → MemRead → MemWB

- 10% stores (STR)
- 13% branches (B)
- 52% R-type (DP)

$$\text{Avg CPI} = (0.13)(3) + (0.52 + 0.10)(4) + (0.25)(5) = 4.12 \tag{6}$$

多周期关键路径

$$T_{c2} = t_{pcq} + 2t_{mux} + \max[t_{ALU} + t_{mux}, t_{mem}] + t_{setup} \tag{7}$$

代入数据（假定 RF 快于 memory，写 memory 快于读 memory）：

$$T_{c2} = 40 + 2(25) + 200 + 50 = 340 \text{ ps} \tag{8}$$

多周期条件逻辑

多周期条件逻辑与单周期的区别：

- PCWrite 在 Fetch 状态断言
- 在 ExecuteI/ExecuteR 状态：CondEx 断言，ALUFlags 产生
- 在 ALUWB 状态：Flags 更新
- CondEx 变化不影响当前正在写的 PCWrite/RegWrite/MemWrite，直到 Fetch 状态才开始新的指令

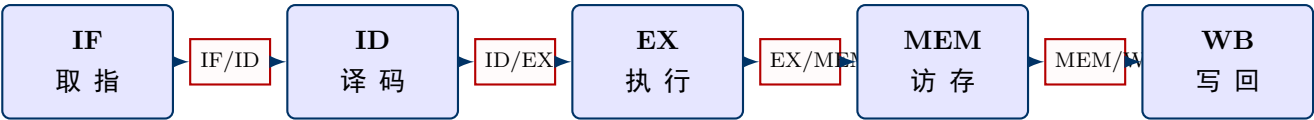
流水线处理器设计 【必考】

流水线基本概念

核心概念 6: 流水线 (Pipelining)

一种时间并行性技术: 将处理器分成 5 个阶段 (Fetch/Decode/Execute/Memory/Writeback), 多个指令在不同阶段同时处理, 提高吞吐率 (Throughput)。

5 级流水线 (IF/ID/EX/MEM/WB)



每个阶段功能:

IF (Instruction Fetch) 从指令存储器取指令, $PC=PC+4$ 。

ID (Instruction Decode) 译码, 读寄存器堆, 扩展立即数。

EX (Execute) ALU 执行运算, 计算 BTA。

MEM (Memory) 读/写数据存储器。

WB (Writeback) 将结果写回寄存器堆。

流水线寄存器: 在各阶段之间插入寄存器组, 保存中间数据 (PC+4、指令、控制信号等), 隔离不同阶段的信号。

流水线吞吐率

核心概念 7: 吞吐率 (Throughput)

$$\text{Throughput} = \frac{\text{完成指令数}}{\text{时间}} = \frac{1}{\text{CPI} \times T_c} \tag{9}$$

理想流水线 $\text{CPI}=1$ (每周周期完成 1 条指令), 吞吐率 $=1/T_c$ 。

流水线 vs 单周期 vs 多周期对比

表 10: 三种实现详细对比 【必考】

指标	单周期	多周期	流水线
CPI	1	4.12	~1.23
T_c	840 ps	340 ps	300 ps
执行时间 (100B)	84 s	140 s	36.9 s
加速比	1	0.6	2.28
硬件利用率	低	中	高
设计复杂度	低	中	高 (需处理冒险)

流水线冒险 (Pipeline Hazards)

核心概念 8: 冒险 (Hazard)

当一条指令依赖尚未完成的指令结果时发生的冲突。

5.5.1 数据冒险 (Data Hazard) —RAW

RAW (Read After Write) 一写后读: 指令 2 需要读一个寄存器, 但指令 1 尚未写回该寄存器。

```
ADD R1, R2, R3    // 在WB阶段才写R1
SUB R4, R1, R5     // 在ID阶段就需要读R1 — 冲突!
```

三种解决数据冒险的方法:

1. 编译时插 NOP: 插入足够 NOP 等待结果就绪 (简单但降低性能)
2. 编译时重排代码: 将不相关指令提前 (不改变语义)
3. 运行时前推 (Forwarding): 将 ALU/存储器结果直接前推到 EX 阶段
4. 运行时停顿 (Stalling): 硬件插入气泡 (bubble) 等待结果

5.5.2 前推 (Data Forwarding) 【必考】

前推 (Forwarding) 绕过寄存器堆, 直接将 EX/MEM 或 MEM/WB 阶段的结果送到 EX 阶段的 ALU 输入。

前推匹配条件 (以 ForwardAE 为例):

- **Match_1E_M:** $RA1E == WA3M$ (EX 阶段读地址与 MEM 阶段写地址匹配)
- **Match_1E_W:** $RA1E == WA3W$ (EX 阶段读地址与 WB 阶段写地址匹配)

前推优先级 (MEM 优先于 WB, 因为数据更新):

```
if (Match_1E_M & RegWriteM)    ForwardAE = 10  // 选择Mem阶段结果
else if (Match_1E_W & RegWriteW) ForwardAE = 01  // 选择WB阶段结果
else                            ForwardAE = 00  // 使用寄存器堆数据
```

ForwardBE 逻辑相同, 使用 Match_2E_M 和 Match_2E_W。

5.5.3 停顿 (Stalling)

前推不能解决 LDR 后立即使用数据的情况 (LDR 数据在 MEM 阶段才读出, 下一条在 EX 阶段就需要)。

LDR 停顿逻辑:

- 检测条件: $Match_12D_E = (RA1D == WA3E) + (RA2D == WA3E)$
- LDR 在 EX 阶段且存在匹配: $ldrstall = Match_12D_E \& MemtoRegE$
- 停顿效果: $StallF = StallD = 1, FlushE = 1$ (将 EX 阶段变为 NOP)

5.5.4 三种数据依赖类型

表 11: 数据依赖 (冒险) 三种类型

类型	含义	说明
RAW (Read After Write)	写后读	真依赖，必须等待。前推 + 停顿解决。
WAW (Write After Write)	写后写	输出依赖（仅乱序执行中出现）。寄存器重命名解决。
WAR (Write After Read)	读后写	反依赖（仅乱序执行中出现）。寄存器重命名解决。

顺序流水线 (5 级) 中只有 RAW 是真正的冒险。WAW 和 WAR 在乱序执行中才会出现！

控制冒险 (Control Hazard) 【必考】

分支指令 B 在 WB 阶段才确定是否跳转，但后续指令已在 IF 阶段被取入——若分支发生，已取的 3 条指令必须被冲刷 (flush)。

分支预测错误惩罚：冲刷的指令数 = 分支到确定时刻的周期数。

提前分支解析 (Early Branch Resolution)：将分支判断提前到 EX 阶段。

- 在 EX 阶段增加分支多路选择器 (Branch Mux)，选择 $BTA=ALUResultE$
- 增加 BranchTakenE 信号（仅当分支条件满足时断言）
- PCSrcW 仅用于写 PC（非分支跳转）
- 惩罚减少为 2 个周期 (ID、EX 各 1 个被冲刷)

控制停顿逻辑：

$$\begin{aligned} PCWrPendingF &= PCSrcD + PCSrcE + PCSrcM \\ StallF &= ldrStallD + PCWrPendingF \\ FlushD &= PCWrPendingF + PCSrcW + BranchTakenE \\ FlushE &= ldrStallD + BranchTakenE \\ StallD &= ldrStallD \end{aligned}$$

流水线 CPI 计算（必考）

SPECINT2000 指令分布 + 额外假设：

- 25% loads, 10% stores, 13% branches, 52% R-type
- 40% loads 被下一条指令使用 (需停顿)
- 50% branches 预测错误 (需冲刷)

$$CPI_{LDR} = 1(0.6) + 2(0.4) = 1.4 \tag{10}$$

$$CPI_B = 1(0.5) + 3(0.5) = 2.0 \tag{11}$$

$$Avg\ CPI = (0.25)(1.4) + (0.1)(1) + (0.13)(2) + (0.52)(1.0) = 1.23 \tag{12}$$

流水线关键路径

流水线周期由最慢的阶段决定：

$$T_{c3} = \max \begin{cases} t_{pcq} + t_{mem} + t_{setup} & (\text{Fetch}) \\ 2(t_{RFread} + t_{setup}) & (\text{Decode}) \\ t_{pcq} + 2t_{mux} + t_{ALU} + t_{setup} & (\text{Execute}) \\ t_{pcq} + t_{mem} + t_{setup} & (\text{Memory}) \\ 2(t_{pcq} + t_{mux} + t_{RFwrite}) & (\text{Writeback}) \end{cases} \quad (13)$$

代入数据，最慢的是 Decode 阶段：

$$T_{c3} = 2(100 + 50) = 300 \text{ ps} \quad (14)$$

高级微体系结构

深度流水线 (Deep Pipelining)

- 现代处理器 10-20 级流水线
- 限制因素：流水线冒险、排序开销、功耗、成本
- 更深流水线 → 更高频率 → 更多冒险和更高的冲刷代价

微操作 (Micro-operations)

核心概念 9: 微操作

将复杂指令分解为一系列简单指令（微操作或 μ -ops），在运行时译码为一或多个微操作。

示例（ARM LDR with writeback）：

原始指令：LDR R1, [R2], #4

微操作序列：

LDR R1, [R2] // 加载

ADD R2, R2, #4 // 基址更新

好处：密集代码 + RISC 简单硬件。

分支预测 (Branch Prediction)

静态预测 (Static) :

- 检查分支方向：向后 (backward)= 跳转 (循环)，向前 (forward)= 不跳转

动态预测 (Dynamic) :

- 维护分支目标缓冲区 (BTB)，记录历史分支目标和是否跳转

1-bit 预测器：记住上次是否跳转，下次做相同预测。对循环：首次和末次预测错误。

2-bit 预测器：4 状态 FSM(强跳转/弱跳转/弱不跳转/强不跳转)。仅末次循环预测错误，优于 1-bit。

超标量处理器 (Superscalar)

核心概念 10: 超标量

多个数据通路副本同时执行多条指令。理想 $IPC > 1$ 。

- 例：双发射处理器理想 $IPC=2$
- 实际 IPC 受限于指令间依赖
- 需要多端口寄存器堆同时提供多个操作数

乱序执行 (Out of Order Execution)

处理器尽可能超前看多条指令，在没有依赖的前提下乱序发射。

记分牌 (Scoreboard)：追踪等待发射的指令、可用的功能单元和依赖关系。

三种依赖类型（仅 OoO 中出现）：

- **RAW**：真依赖，必须等待
- **WAW**：输出依赖，通过寄存器重命名解决
- **WAR**：反依赖，通过寄存器重命名解决

指令级并行 (ILP)：可同时发射的指令数，平均值 < 3 。

寄存器重命名 (Register Renaming)

通过将架构寄存器映射到更大的物理寄存器池，消除 WAW 和 WAR 伪依赖。

```
LDR R8, [R0, #40]    // R8 -> P1
ADD R9, R8, R1        // R8 -> P1 (真依赖)
SUB R8, R2, R3        // R8 -> P2 (重命名, 消除WAW!)
AND R10, R4, R8       // R8 -> P2 (真依赖)
```

重命名后 IPC 从 $6/4=1.5$ 提升至 $6/3=2$ 。

SIMD (单指令多数据)

核心概念 11: SIMD

单条指令同时作用于多个数据。常见应用：图形处理 (如一次加 8 个 8 位元素)。

多线程与多处理器

多线程 (Multithreading) : 多份架构状态副本, 多线程同时活跃。Intel 称为超线程 (Hyperthreading)。

多处理器 (Multiprocessors) : 单芯片上多个处理器 (核), 通过共享内存或消息通信。

同构多核 : 多个相同核共享主存。

异构多核 : 不同类型核处理不同任务 (如手机中 DSP+CPU)。

考试重点提示 【必考】（考前必看!）

★★★★★必考 1：单周期 CPU 设计（约 30–40 分）

- 必考填空：控制信号真值表 (Main Decoder/ALU Decoder) — 给定指令类型，填写 MemtoReg, MemW, ALUSrc, ImmSrc, RegW, ALUOp 等信号值
- 必考填空：给 ImmSrc 编码，写出对应的立即数扩展方式 (8→32 零扩/12→32 零扩/24→32 符扩)
- 必考简答：LDR 指令在单周期 CPU 上的执行过程逐步描述 (6 步)
- 必考简答：STR 和 LDR 数据通路的区别 (多了一个读口、MemW 信号)
- 可能考：结构化建模 vs 行为建模：顶层模块如何实例化子模块
- 可能考：FlagW[1:0] 含义：NZ vs CV 标志的更新规则
- 可能考：单周期关键路径分析，给定延迟参数计算 T_c

★★★★★必考 2：流水线基本概念（约 20–30 分）

- 必考简答：解释流水线概念、5 级流水线各阶段功能 (IF/ID/EX/MEM/WB)
- 必考简答：单周期 vs 多周期 vs 流水线的区别 (CPI/周期时间/吞吐率对比)
- 必考填空/计算：流水线吞吐率公式 $\text{Throughput} = 1/(\text{CPI} \times T_c)$
- 必考简答：数据冒险是什么？三种解决方式 (前推/停顿/NOP)
- 必考填空：前推匹配条件 Match_1E_M / Match_1E_W / ForwardAE 编码 (00/01/10)
- 必考简答：控制冒险是什么？提前分支解析 (Early Branch Resolution) 如何减少惩罚？
- 必考计算：给定指令分布和停顿/预测错误比例，计算流水线 CPI
- 可能考：LDR 后相随指令为什么前推无法解决，需要停顿？
- 可能考：分支预测错误惩罚：提前到 EX 阶段后惩罚 = 2 周期 (无提前 = 3 周期)

控制器/数据通路填空题（约 10–20 分）

- 必考填空：多路选择器控制信号：ALUSrc 选择寄存器还是立即数
- 必考填空：RegSrc[1:0] 选择寄存器堆读口地址
- 必考填空：MemtoReg 选择写回 ALUResult 还是 MemData
- 必考填空：条件标志位改写时机：FlagWrite 仅当 FlagW 不为 0 且 CondEx=1 时有效
- 必考填空：PCSrc 何时 = 1？ $((\text{Rd}==15)\&\text{RegW})|\text{Branch}$
- 必考填空：多周期各指令的周期数：B=3, DP=4, STR=4, LDR=5

性能计算（约 10 分）

- $\text{Execution Time} = \# \text{instructions} \times \text{CPI} \times T_c$
- 单周期 $T_c = 840\text{ps}$, $\text{CPI} = 1$
- 多周期 $T_c = 340\text{ps}$, $\text{CPI} = 4.12 = (0.13)(3) + (0.52 + 0.1)(4) + (0.25)(5)$
- 流水线 $T_c = 300\text{ps}$, $\text{CPI} = 1.23$
- 三种实现对比：流水线 > 单周期 > 多周期 (加速比: $2.28 > 1 > 0.6$)
- 注意多周期虽然 T_c 更小但总时间可能更长! (考试陷阱)

考试必背公式

1. 执行时间 = $\#instructions \times CPI \times T_c$
2. 吞吐率 = $1/(CPI \times T_c)$
3. $IPC = 1/CPI$
4. $BTA = (ExtImm24 \ll 2) + (PC + 8)$
5. $PCS = ((Rd == 15) \& RegW) | Branch$
6. 多周期平均 $CPI = \sum(\text{各类指令占比} \times \text{该类 CPI})$
7. 流水线平均 $CPI = \text{各指令 CPI 的加权平均 (考虑停顿和冲刷)}$
8. 加速比 = 基准执行时间 / 优化后执行时间

考前提醒

- 所有控制信号真值表**必须默写**！
- LDR 指令 6 步执行过程**必须能逐步描述**！
- 5 级流水线各阶段**必须说出名称和功能**！
- 三种冒险及对应解决方案**必须区分清楚**！
- 性能公式三个要素关系**必须理解**（CPI 减少不一定总执行快，见多周期反例）！
- 三个 T_c 值、三个 CPI 值**必须会算**！

附录：综合自测题

1. 【控制信号填空】对于指令 STR R3, [R5, #8], 填写控制信号值:

MemtoReg	MemW	ALUSrc	ImmSrc	RegW	RegSrc	ALUOp	Branch
X	1	1	01	0	10	0	0

2. 【立即数扩展】B LOOP 指令的 ImmSrc 和扩展方式:

ImmSrc=10, sign-extend {6{Instr[23]}, Instr[23:0]}, 左移 2 位 → 26 位偏移。

3. 【关键路径计算】给定延迟数据, 求单周期 T_c :

答案: $T_{c1} = t_{pcq_PC} + t_{mem} + t_{dec} + t_{RFread} + t_{sext} + t_{mux} + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup} = 840 \text{ ps}$

4. 【多周期 CPI】指令分布: 25% load, 10% store, 13% branch, 52% DP。求多周期 CPI。

答案: $\text{Avg CPI} = (0.25 \times 5) + (0.1 \times 4) + (0.13 \times 3) + (0.52 \times 4) = 4.12$

5. 【流水线 CPI】额外条件: 40% load 被下条使用, 50% branch 预测错。求流水线 CPI。

答案: $\text{Avg CPI} = (0.25 \times 1.4) + (0.1 \times 1) + (0.13 \times 2) + (0.52 \times 1) = 1.23$

6. 【前推编码】若 EX 阶段 RA1=WB 阶段 WA3 且 RegWriteW=1, ForwardAE=?

答案: ForwardAE = 01 (选择 WB 阶段前推数据)

7. 【冒险识别】SUB R1, R2, R3 后紧跟 ADD R4, R1, R5, 是否需前推?

答案: 需要。R1 在 SUB 的 WB 阶段才写入, ADD 在 EX 阶段就需读取。可从 EX/MEM 前推 (ForwardAE=10)。